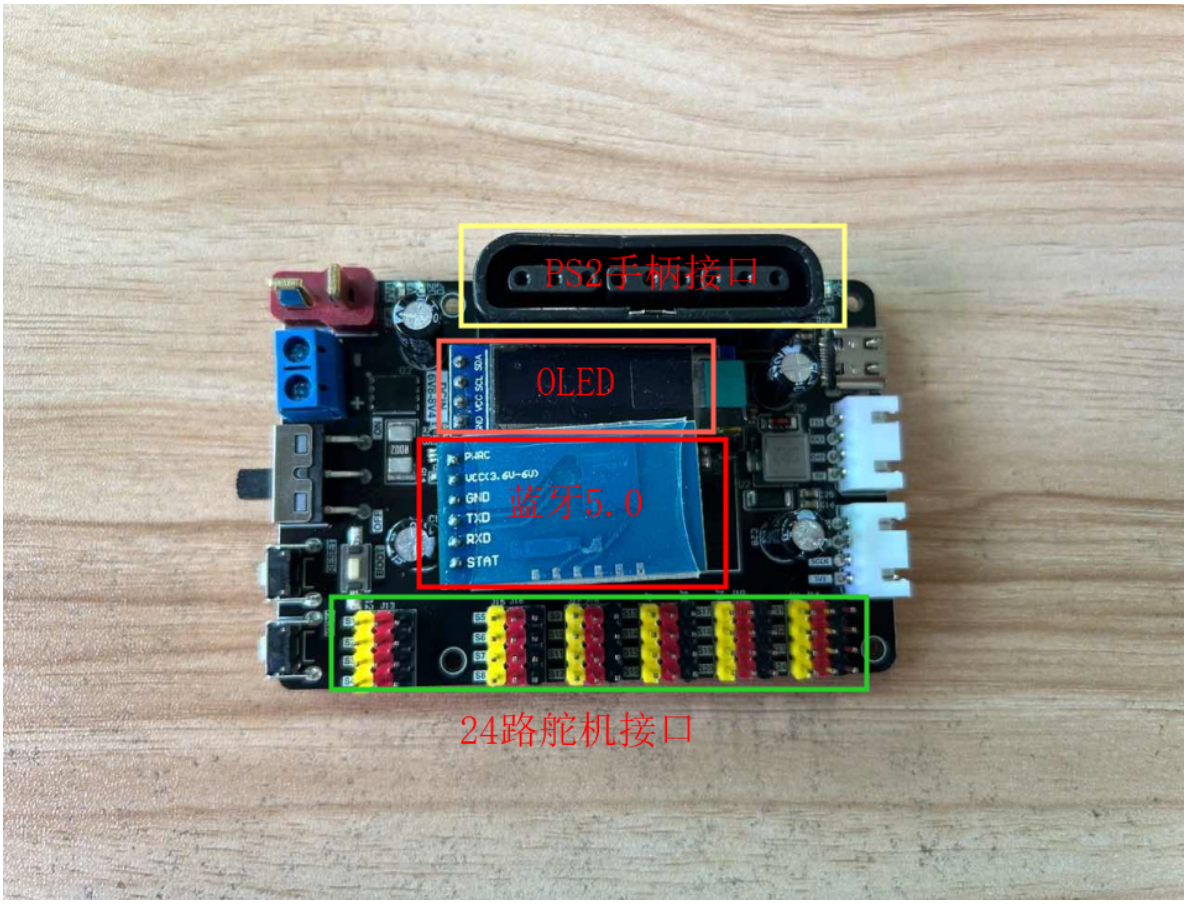
24路pwm舵机控制板

24路pwm舵机控制板
- 1.硬件准备
- 2.主要的硬件介绍
  - 2.1 24路舵机控制板
  - 2.2 蓝牙模块的介绍
  - 2.3 Ps2手柄介绍
- 3.实验目的
- 4.实验步骤
  - 4.1看懂原理图
  - 4.2 ps2手柄控制说明
  - 4.3 手机app控制说明
  - 4.4 舵机的运动
- 5.主要的源码分析
- 6.实验总结
- 7.拓展部分和板子的特色功能
  - 1.拓展部分-精准控制270度舵机
  - 2.特色功能

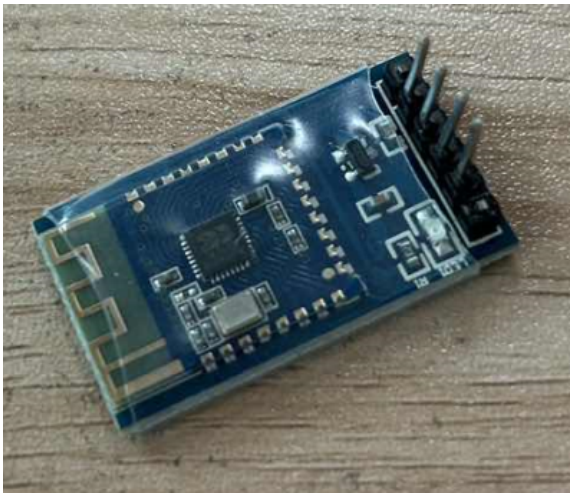
1.硬件准备

*** 24路舵机控制板一个 ***

连上一部分的模块后，如图：



*** 蓝牙模块(5.0)一个 ***



*** ps2手柄 ***

*** 7.4v电池一个 ***

2.主要的硬件介绍

2.1 24路舵机控制板

该控制板使用的控制芯片是STM32RCT6,具有32*128的oled接口、4.0蓝牙接口、ps2手柄接口以及24路pwm舵机的接口

2.2 蓝牙模块的介绍

蓝牙模块是专为智能无线数据传输而打造，采用美国 TI 公司 CC2541 芯片，配置 256Kb 空间，遵循 V5.0 BLE 蓝牙规范。支持 AT 指令，用户可根据需要更改串口波特率、设备名称、配对密码等参数，使用灵活.
本模块支持 UART 接口，并支持 SPP 蓝牙串口协议，具有成本低、体积小、功耗低、收发灵敏性高等优点，只需配备少许的外围元件就能实现其强大功能。
具有以下的优点

- 支持标准 BLE 协议；
- 支持 UART，I2C 接口；
- 支持低功耗模式；
- 支持蓝牙 Class1 和 Class2 模式；
- 工业级设计；
- 数据加密；
- 内置 PCB 天线；
- 输入电压 3V；

2.3 Ps2手柄介绍

① DI/DAT ：信号流向，从手柄到主机，此信号是一个 8bit 的串行数据，同步传送于时钟的下降沿。信号的读取在时钟由高到低的变化过程中完成。

②DO/CMD：信号流向，从主机到手柄，此信号和 DI 相对，信号是一个 8bit 的串行数据，同步传送于时钟的下降沿。

④GND：电源地。

⑤VDD：接收器工作电源，电源范围 3~5V。

⑥CS/SEL：用于提供手柄触发信号。在通讯期间，处于低电平。

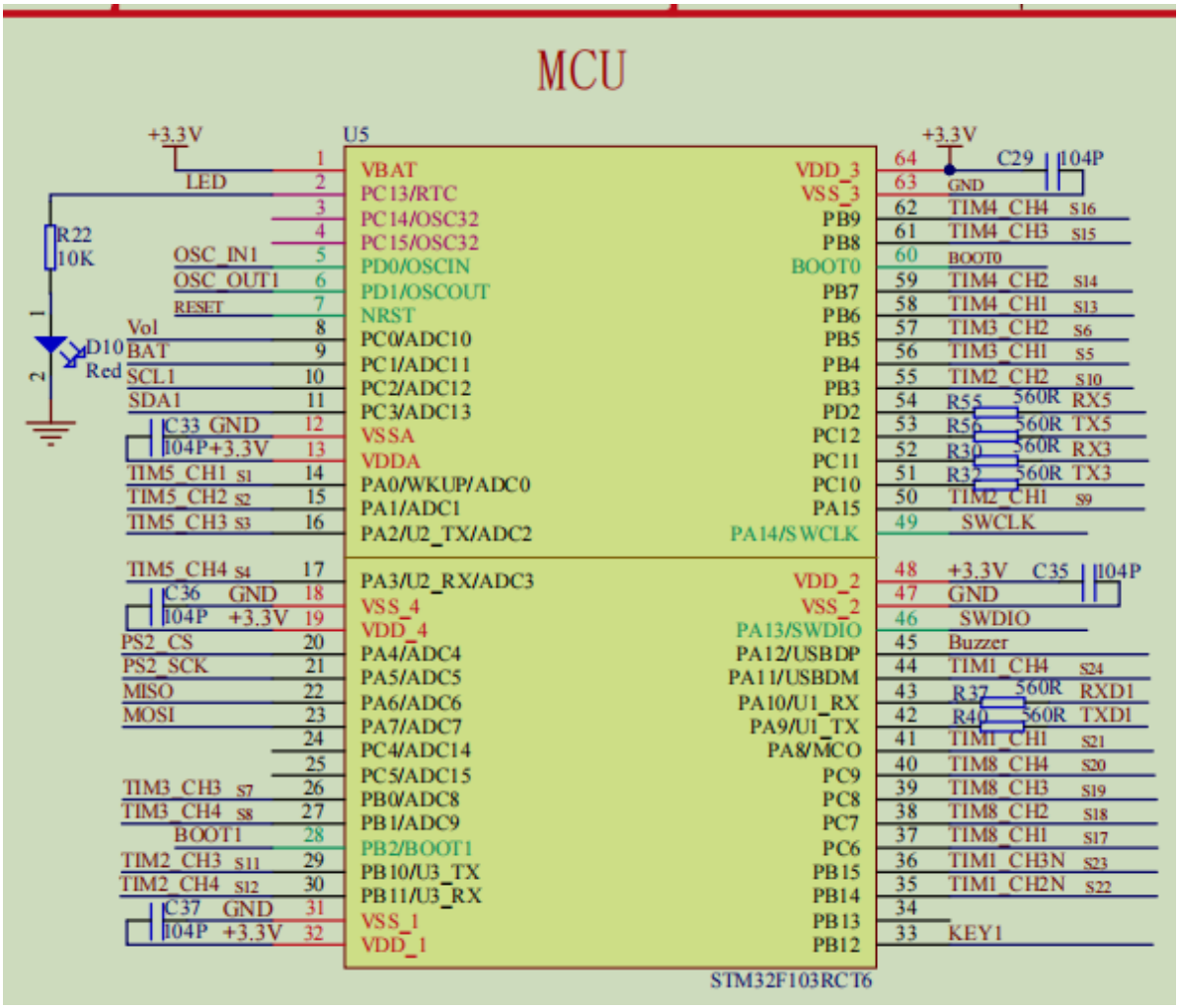
⑦CLK：时钟信号，由主机发出，用于保持数据同步。

3.实验目的

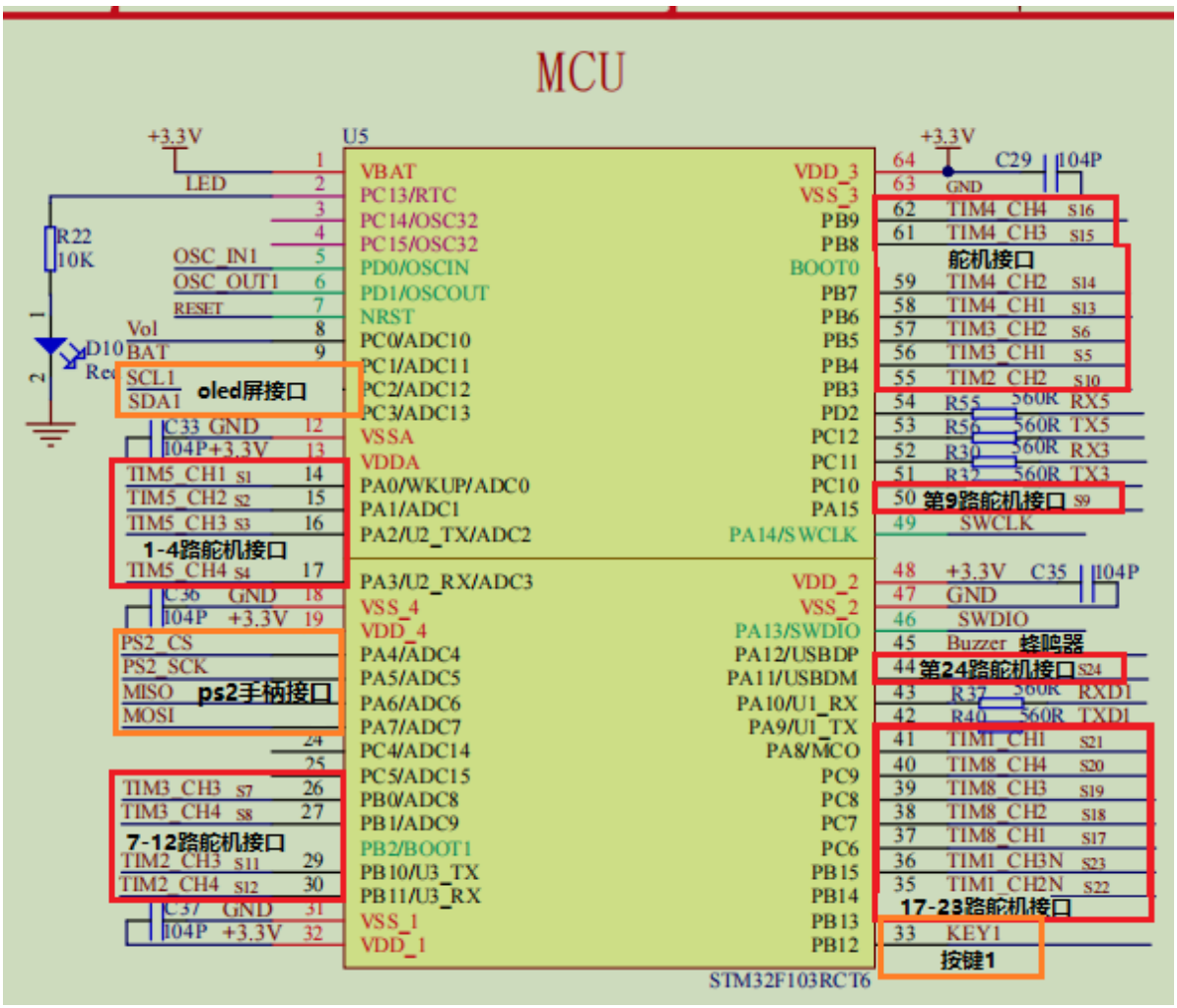
通过24路舵机控制板控制舵机运动，实现手机app蓝牙控制、ps2手柄控制的过程，以致于了解舵机的工作原理。

4.实验步骤

4.1看懂原理图



由原理图我们可以清楚的了解stm32各个引脚是如何控制24路舵机控制板的。



- 串口5是和蓝牙进行通信的
- 串口3是留出来与树莓派、jetson系列上位机进行通信的。

4.2 ps2手柄控制说明

根据原理图，对stm32的引脚做相对应的初始化
手柄接口初始化：
输入 DI->PA6
输出 DO->PA7 CS->PA4 CLK->PA5
再根据ps2手柄的工作原理，编写相关的驱动。
工作原理相关的参考连接：
https://zhuanlan.zhihu.com/p/369859276

*** 手柄控制原理 ***
手柄控制只能一次控制8路舵机，因此本实验把24路舵机分成3组，每组有8路舵机，使用了板上的按键1进行切换舵机组数的切换（按键1的值：0->1->2->0），实现手柄控制24路舵机的功能。

*** 手柄控制现象 ***
通过手柄上的每两个按键（一个增大角度、一个减小角度）对应着一路的舵机控制
按键和舵机对应的如下：

增大角度	减小角度	按键key值	舵机
SELECT	START	0	S1路舵机
L3	R3		S2路舵机

增大角度	减小角度	按键key值	舵机
up	DOWN		S3路舵机
RIGHT	LEFT		S4路舵机
L2	R2		S5路舵机
L1	R1		S6路舵机
GREEN	BLUE		S7路舵机
RED	PINK		S8路舵机
SELECT	START	1	S9路舵机
L3	R3		S10路舵机
up	DOWN		S11路舵机
RIGHT	LEFT		S12路舵机
L2	R2		S13路舵机
L1	R1		S14路舵机
GREEN	BLUE		S15路舵机
RED	PINK		S16路舵机
SELECT	START	2	S17路舵机
L3	R3		S18路舵机
up	DOWN		S19路舵机
RIGHT	LEFT		S20路舵机
L2	R2		S21路舵机
L1	R1		S22路舵机
GREEN	BLUE		S23路舵机
RED	PINK		S24路舵机

4.3 手机app控制说明

手机app是通过蓝牙和24路舵机控制板上的蓝牙模块进行通信的，从而间接控制舵机的运动。
通信的协议如下：

串口通讯(波特率 9600)				
	包头	舵机编号	舵机角度	包尾
数据	‘$’	‘A-P’	‘0-180’	‘#’
例子	舵机 1 角度 10 度：$A010#			

安装我们提供的app,在app界面连接好蓝牙，即可实现手机app控制舵机转动的目标。

app的下载二维码如图：

在APP界面连接好蓝牙后，进入舵机控制的页面

4.4 舵机的运动

本教程采用的是定时器模拟pwm的方式来实现舵机的角度控制的，主要的代码如下：（以一路pwm舵机为例）

```
void TIM1_UP_IRQHandler(void)    //TIM1中断
{
  if (TIM_GetITStatus(TIM1, TIM_IT_Update) != RESET)  //检查TIM1更新中断发生与否
  {
    TIM_ClearITPendingBit(TIM1, TIM_IT_Update);  //清除TIM1更新中断标志
    num++;


    #ifdef USE_SERVO_J1
    if(num <= (Angle_J[0] * 11 + 500)/10)
    {
      GPIO_SetBits(Servo_J1_PORT, Servo_J1_PIN );   //将舵机接口电平置高
    }
    else
    {
      GPIO_ResetBits(Servo_J1_PORT, Servo_J1_PIN );   //将舵机接口电平置高
    }
    #endif
if(num == 2000) //2000*10=20ms  20ms一个周期
    {
      num = 0;
    }

  }
}
```

5.主要的源码分析

```
int main(void)
{
    u8 Dnum;
    u8 Doup;
    //舵机角度初始化
    for(Doup = 0;Doup <GROUP_NUM;Doup++)
    {
        for(Dnum = 0;Dnum <DUOJI_NUM;Dnum++)
        {
            Angle_J[Doup][Dnum] = 90;
        }

    }

    bsp_init();//再进行硬件初始化
    while (1)
    {
        dectect_beep();//检测电压
        void_jutce(); //检测舵机总电流
        user_douji();   //手柄控制
        if(Key1_State(1)==1)//每按一下只生效一次
        {
            grop++;
            if(grop>2)
            {
                grop = 0;
            }
```

```
        }
    }

}
```

- Angle_J：是控制舵机角度的一个二维数组
- GROUP_NUM：是定义把24路舵机分成几组
- DUOJI_NUM：是定义的有每一组舵机的数量
- 定时器的方式控制舵机角度，则是根据Angle_J[0][0]这个数值控制第一路舵机的角度，控制其它路的舵机同理。
- bsp_init：该函数是根据原理图，对用到的IO口做相对应的初始化，具体的内容看源码。
- dectect_beep：该函数是检测电池是否有6.5V以上的电源，低于6.5V会自动报警（蜂鸣器鸣叫），只能通过更换6.5V以上的电源才能解除警报。
- void_jutce: 该函数是检测舵机的总电流是否大于16A,如果是则说明舵机过载，蜂鸣器会立即鸣叫。
- user_douji：该函数是实现手柄的一个控制，具体的内容看源码。
- 手机app的控制，则是采用了串口中断的方式进行控制的，中断处理的源码在bsp_usart.c 串口5的内容
- 在 AllHeader.h 文件中还定义的一些外设开关，如果没连接一些外设的情况下，可以把这些外设功能关了，减少功耗和内存，提升速度，特别是没接ps2手柄的情况下，记得一定要把手柄的功能关了，不然会影响判断。

6.实验总结

通过实验和源码分析，可以清楚的了解到舵机的工作原理如下：
舵机工作原理：控制信号由接收机的通道进入信号调制芯片，获得直流的偏置电压。它内部有一个基准电路，产生周期为20ms，宽度为1.5ms的基准信号，将获得的直流偏置电压与电位器的电压比较，获得电压差输出。最后电压差的正负输出到电机驱动芯片决定电机的正反转。当电机转速一定时，通过级联减速此轮带动电位器旋转，使得电压差为0，电机停止转动。
舵机的控制：一般需要一个20ms左右的时基脉冲，该脉冲的高电平部分一般为0.5ms-2.5ms范围内的角度控制脉冲部分。本次实验的采用的舵机是180度伺服，控制关系如下：

0.5ms----------0度

1.0ms----------45度

1.5ms----------90度

2.0ms----------135度

2.5ms----------180度

7.拓展部分和板子的特色功能

1.拓展部分-精准控制270度舵机

- 教程的源码同样适用于驱动270度舵机，在本教程里270度舵机控制的角度并不准确，因为控制的公式还是使用控制180度舵机的公式。
- 需要精准控制270度舵机，可以把源码的bsp_timer.c文件里面的中断函数进行改写。如：第一路舵机控制是个270度舵机，则在TIM1_UP_IRQHandler()函数里改写第一路舵机的控制公式：

```
//把这个公式（180度）
num <= (Angle_J[0][0] * 11 + 500)/times
//改成270度的公式，即：
num <= (Angle_J[0][0] * 8 + 500)/times
```

控制角度的范围就会从0-180度变成0-270度了。
对应的关系如图所示：

输入信号脉冲宽度 (ms)	0.5	0.83	1.16	1.5	1.83	2.16	2.5
舵机输出角度	0	45	90	135	180	225	270

2.特色功能

2.1 板子具有检测舵机转动时的电流功能，此功能具有板子的保护、舵机发生电流过载的报警作用。

2.2 板子具有实时检测电池的电压，并显示在oled屏上

代码如下：

```
char buf[20]={'\0'};
void dectect_beep(void)
{
  u8 V_flag = 1;//电压
  float vol;
  V_flag = Bsp_Bat_State();

  if(V_flag == 0)//低压的情况
  {
    set_beep(5);//不停的响
    OLED_Draw_Line("voltage low...", 1, false, true);
  }

  if(V_flag == 1)//正常的情况
  {
    vol = Bsp_Bat_Voltage_Z100()/100.0;
    sprintf(buf," vol is  %.1f V ",vol);
    OLED_Draw_Line(buf, 3, false, true);
  }


  beep_start_stop();//蜂鸣器操作
}

// 查询电池电压状态，连续几秒读到低于6.5V返回0，高于6.5V返回1
u8 Bsp_Bat_State(void)
{
  if (g_bat_state)
  {
    Voltage_Z100 = (int) (Adc_Get_Battery_Volotage() * 100);
    if (Voltage_Z100 < 650)
    {
      Voltage_Low_Count++;
      if(Voltage_Low_Count > BAT_CHECK_COUNT)
      {
        g_bat_state = 0;
      }
    }
    else
    {
      Voltage_Low_Count = 0;
    }
  }
  return g_bat_state;
}
```

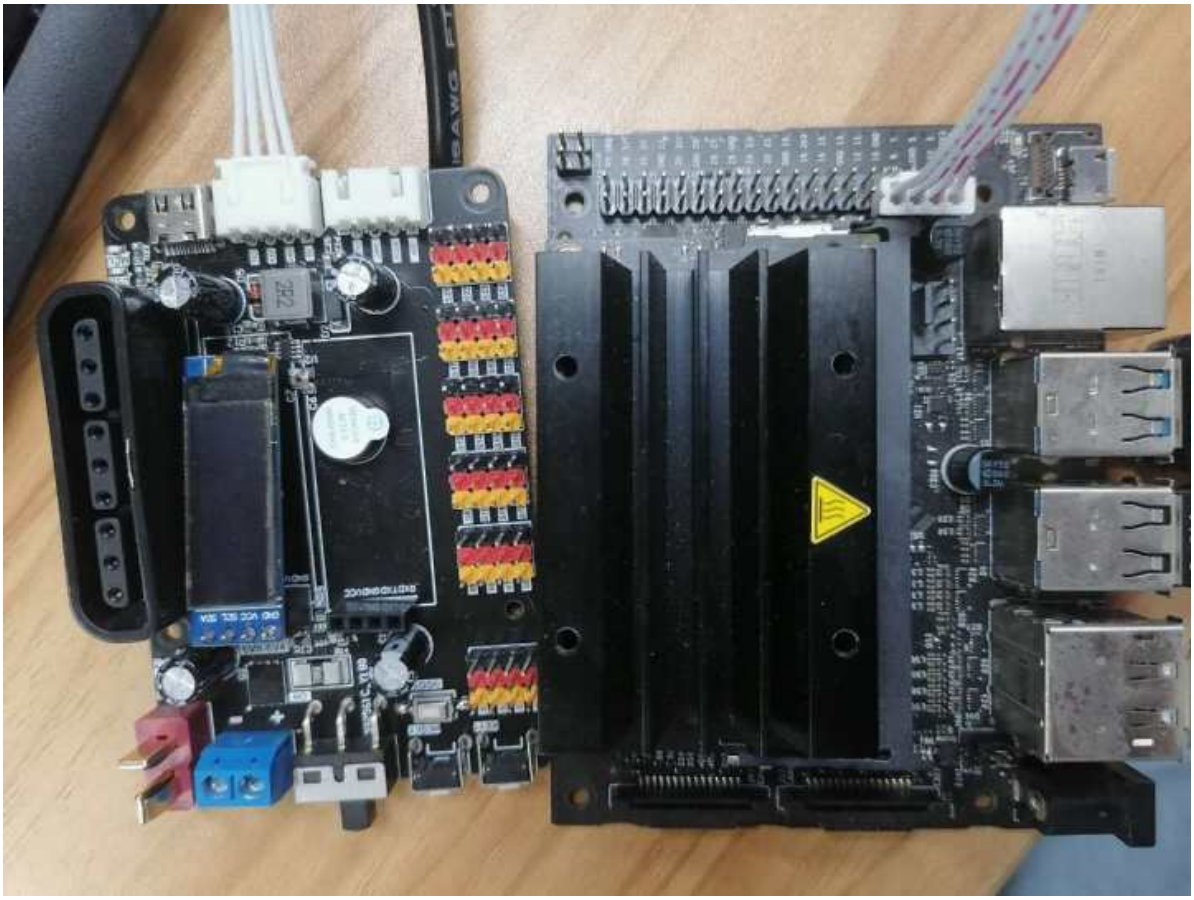
由上述代码可知：该函数是检测电池是否有6.5V以上的电源，低于6.5V会自动报警（蜂鸣器鸣叫），只能通过更换6.5V以上的电源才能解除警报

2.3 板子的串口有着能和树莓派、jetson nano引脚上串口进行直连，可以很大程度上避免接线的繁琐，大大提高接线效率。
如图所示：

舵机控制板与树莓派的接线图
碰到程序报错，可以参考下一个教程文档

```
#碰到找不到模块的错误可以使用以下命令进行安装
sudo pip install serial
sudo pip install pyserial
```

或者其它参考链接：https://blog.iyatt.com/?p=1817



舵机控制板与jetson nano的接线图
碰到程序报错，可以参考下一个教程文档
或者其它参考链接：https://blog.iyatt.com/blog.iyatt.com/?p=6661

- 在程序源码的test文件夹中，有着树莓派、jetson nano和舵机控制板的串口通信的程序。

*** 程序现象 ***
树莓派/jetson nano 通过串口不停的发送'H'这个字母，舵机控制板收到这信息会马上返回收到的信息，即发什么回什么。